გაკვეთილი 26 mod 3 (3.5)

გაკვეთილი 1 — DOM-ის საფუძვლები და HTML ელემენტების მოძიება

1. თეორიული მასალა

1.1. რა არის DOM?

DOM (Document Object Model) არის დოკუმენტის ობიექტური მოდელი, რომლის დახმარებითაც JavaScript-ს შეუძლია HTML დოკუმენტთან ურთიერთქმედება.

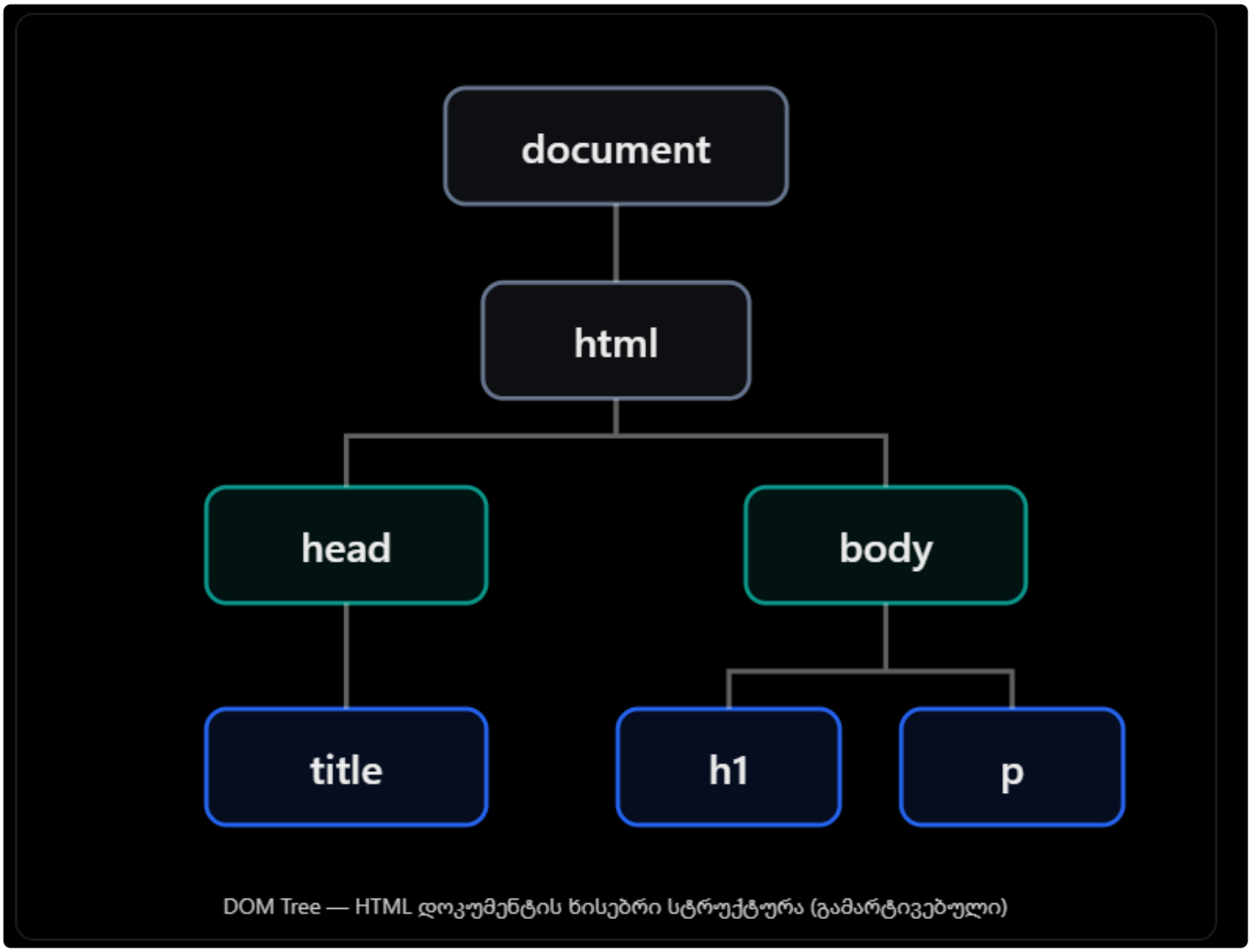
როდესაც ბრაუზერი ხსნის HTML გვერდს, ის HTML კოდის საფუძველზე ქმნის DOM-ის სტრუქტურას.

DOM-ში HTML ელემენტები წარმოდგენილია ობიექტების სახით, რომელთა მოძიება, შეცვლა, დამატება და წაშლა შესაძლებელია JavaScript-ით.

მაგალითი:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>My Website</title>
</head>
<body>
    <h1>Hello World</h1>
    <p>Welcome to my website</p>
</body>
</html>
```

ბრაუზერი ამ HTML კოდის საფუძველზე ქმნის შემდეგ ხისებრ სტრუქტურას:



DOM Tree — HTML დოკუმენტის ხისებრი სტრუქტურა (გამარტივებული)

DOM-ის ძირითადი ცნებები:

- Document — მთლიანი HTML დოკუმენტი.
- Element — HTML ელემენტი, მაგალითად `h1`, `p`, `div`.
- Parent — მშობელი ელემენტი, რომელიც შეიცავს სხვა ელემენტებს.
- Child — შვილობილი ელემენტი, რომელიც მოთავსებულია სხვა ელემენტის შიგნით.

მაგალითად, `body` არის `h1`-ისა და `p`-ის მშობელი ელემენტი, ხოლო `h1` და `p` მისი შვილობილი ელემენტებია.

1.2. JavaScript-ის დაკავშირება HTML დოკუმენტთან

JavaScript-ს HTML ელემენტებთან სამუშაოდ ჯერ უნდა ჰქონდეს წვდომა HTML დოკუმენტზე.

ამისთვის HTML-ში შეგვიძლია გამოვიყენოთ `<script>` ტეგი.

მაგალითი 1 — JavaScript უშუალოდ HTML-ში:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>DOM Example</title>
</head>
<body>
    <h1>Hello!</h1>

    <script>
        console.log("JavaScript is working!");
    </script>
</body>
</html>
```

მაგალითი 2 — გარე JavaScript ფაილის დაკავშირება:

HTML ფაილი — `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>DOM Example</title>
    <script src="script.js" defer></script>
</head>
<body>
    <h1>Hello World</h1>
    <p>This is my website.</p>
</body>
</html>
```

JavaScript ფაილი — `script.js`

```
console.log("JavaScript connected!");
```

აქ `defer` უზრუნველყოფს, რომ გარე JavaScript ფაილი შესრულდეს HTML დოკუმენტის დამუშავების დასრულების შემდეგ. ეს მნიშვნელოვანია, რადგან JavaScript-მა უნდა შეძლოს საჭირო HTML ელემენტების მოძიება.

1.3. `document` ობიექტი

`document` არის JavaScript-ის ობიექტი, რომელიც წარმოადგენს ბრაუზერში გახსნილ HTML დოკუმენტს.

მისი დახმარებით შესაძლებელია HTML ელემენტების მოძიება და მათთან მუშაობა.

მაგალითი:

```
console.log(document);
```

ეს ბრძანება ბრაუზერის კონსოლში გამოიტანს დოკუმენტის ობიექტს.

```
console.log(document.title);
```

ეს ბრძანება გამოიტანს გვერდის სათაურს, რომელიც `<title>` ტეგში წერია.

```
console.log(document.body);
```

ეს ბრძანება გამოიტანს HTML დოკუმენტის `body` ელემენტს.

მაგალითი:

```
<title>My First Website</title>
```

```
console.log(document.title);
```

შედეგი:

```
My First Website
```

2. HTML ელემენტების მოძიების მეთოდები

JavaScript-ში HTML ელემენტების მოსაძებნად რამდენიმე მეთოდი არსებობს.

დღევანდელ გაკვეთილზე განვიხილავთ სამ ძირითად მეთოდს:

1. `getElementById()`
2. `getElementsByClassName()`
3. `getElementsByTagName()`

2.1. `getElementById()`

`getElementById()` გამოიყენება HTML ელემენტის მოსაძებნად მისი `id` ატრიბუტის მიხედვით.

HTML-ში `id` უნდა იყოს უნიკალური, ანუ ერთ დოკუმენტში ერთი და იგივე `id` სხვადასხვა ელემენტს არ უნდა ჰქონდეს.

მაგალითი:

HTML:

```
<h1 id="title">Hello World</h1>
<p id="description">This is my website.</p>
```

JavaScript:

```
let heading = document.getElementById("title");

console.log(heading);
```

შედეგი:

```
<h1 id="title">Hello World</h1>
```

განმარტება:

- `document` — HTML დოკუმენტი.
- `getElementById("title")` — ეძებს ელემენტს, რომელსაც აქვს `id="title"`.
- `heading` — ცვლადი, რომელშიც ინახება მოძიებული ელემენტი.
- `console.log()` — გამოაქვს მოძიებული ელემენტი კონსოლში.

კიდევ ერთი მაგალითი:

```
<h2 id="name">Giorgi</h2>
<p id="age">20</p>
```

```
let userName = document.getElementById("name");
let userAge = document.getElementById("age");

console.log(userName);
console.log(userAge);
```

ამ შემთხვევაში JavaScript მოძებნის ორივე ელემენტს და გამოიტანს კონსოლში.

თუ მითითებული `id` არ არსებობს, მეთოდი დააბრუნებს `null` მნიშვნელობას.

2.2. `getElementsByClassName()`

`getElementsByClassName()` გამოიყენება HTML ელემენტების მოსაძებნად მათი `class` ატრიბუტის მიხედვით.

ერთი და იგივე კლასი შეიძლება რამდენიმე HTML ელემენტს ჰქონდეს.

მაგალითი:

HTML:

```
<p class="text">First paragraph</p>
<p class="text">Second paragraph</p>
<p class="text">Third paragraph</p>
```

JavaScript:

```
let paragraphs = document.getElementsByClassName("text");

console.log(paragraphs);
```

მეთოდი აბრუნებს `HTMLCollection`-ს, რომელიც შეიცავს შესაბამის ელემენტებს.

ამ კოლექციის ელემენტებთან მუშაობა შეგვიძლია ინდექსების გამოყენებით.

```
console.log(paragraphs[0]);
console.log(paragraphs[1]);
console.log(paragraphs[2]);
```

ინდექსაცია იწყება 0-დან.

ინდექსი	შესაბამისი ელემენტი
0	First paragraph
1	Second paragraph
2	Third paragraph

ელემენტების რაოდენობის გაგება:

```
console.log(paragraphs.length);
```

შედეგი:

```
3
```

`length` გვიჩვენებს, რამდენი ელემენტია კოლექციაში.

მაგალითი — ყველა ელემენტის გამოტანა ციკლით:

```
let paragraphs = document.getElementsByClassName("text");

for (let i = 0; i < paragraphs.length; i++) {
    console.log(paragraphs[i]);
}
```

ამ კოდის დახმარებით JavaScript სათითაოდ გამოიტანს ყველა მოძიებულ პარაგრაფს.

მნიშვნელოვანია: `HTMLCollection` ჩვეულებრივი მასივი არ არის, თუმცა აქვს ინდექსები და `length` თვისება.

2.3. `getElementsByTagName()`

`getElementsByTagName()` გამოიყენება HTML ელემენტების მოსაძებნად მათი ტეგის სახელის მიხედვით.

მაგალითად, შეგვიძლია მოვძებნოთ ყველა `p`, `h1`, `div` ან `li` ელემენტი.

მაგალითი:

HTML:

```
<h1>My Website</h1>

<p>First paragraph</p>
<p>Second paragraph</p>
<p>Third paragraph</p>
```

JavaScript:

```
let paragraphs = document.getElementsByTagName("p");

console.log(paragraphs);
```

ეს ბრძანება მოძებნის ყველა `p` ელემენტს.

კონკრეტული ელემენტის მიღება:

```
console.log(paragraphs[0]);
console.log(paragraphs[1]);
```

ყველა ელემენტის გამოტანა ციკლით:

```
let paragraphs = document.getElementsByTagName("p");

for (let i = 0; i < paragraphs.length; i++) {
    console.log(paragraphs[i]);
}
```

კიდევ ერთი მაგალითი:

HTML:

```
<ul>
    <li>Apple</li>
    <li>Banana</li>
    <li>Orange</li>
    <li>Grape</li>
</ul>
```

JavaScript:

```
let fruits = document.getElementsByTagName("li");

console.log(fruits.length);
console.log(fruits[0]);
console.log(fruits[3]);
```

შედეგად გამოვა სიის ელემენტების რაოდენობა, პირველი ელემენტი და მეოთხე ელემენტი.

3. მეთოდების შედარება

მეთოდი	რას ეძებს?	რას აბრუნებს?
`getElementById()`	კონკრეტული `id`-ის მქონე ელემენტს	ერთ ელემენტს ან `null`-ს
`getElementsByClassName()`	კონკრეტული კლასის მქონე ელემენტებს	`HTMLCollection`-ს

მეთოდი	რას ეძებს?	რას აბრუნებს?
`getElementsByTagName()`	კონკრეტული ტეგის მქონე ელემენტებს	`HTMLCollection`-ს

დასამახსოვრებელი:

- `getElementById()` გამოიყენება ერთი კონკრეტული ელემენტის მოსაძებნად.
- `getElementsByClassName()` გამოიყენება ერთი კლასის მქონე ელემენტების მოსაძებნად.
- `getElementsByTagName()` გამოიყენება ერთი ტიპის HTML ტეგების მოსაძებნად.

4. სრული პრაქტიკული მაგალითი

შევქმნათ ვებგვერდი, სადაც გვაქვს სათაური, რამდენიმე პარაგრაფი და სია.

HTML — `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>DOM Practice</title>
    <script src="script.js" defer></script>
</head>
<body>

    <h1 id="main-title">My Website</h1>

    <p class="description">First description</p>
    <p class="description">Second description</p>

    <h2>My Favorite Languages</h2>

    <ul>
        <li>JavaScript</li>
        <li>Python</li>
        <li>Java</li>
    </ul>

</body>
</html>
```

JavaScript — `script.js`

```
// 1. სათაურის მოძიება ID-ით
let title = document.getElementById("main-title");

console.log(title);

// 2. პარაგრაფების მოძიება კლასით
let descriptions = document.getElementsByClassName("description");

console.log(descriptions);

// 3. პარაგრაფების რაოდენობის გამოტანა
console.log(descriptions.length);

// 4. პირველი პარაგრაფის გამოტანა
console.log(descriptions[0]);

// 5. სიის ელემენტების მოძიება
let items = document.getElementsByTagName("li");

// 6. ყველა სიის ელემენტის გამოტანა
for (let i = 0; i < items.length; i++) {
    console.log(items[i]);
}
```

ამ მაგალითში JavaScript მხოლოდ პოულობს ელემენტებს და გამოაქვს კონსოლში. ელემენტების ტექსტისა და შიგთავსის შეცვლას მომდევნო გაკვეთილზე ვისწავლით.

5. პრაქტიკული სავარჯიშოები

მოსწავლეებმა დამოუკიდებლად უნდა შექმნან HTML და JavaScript ფაილები და შეასრულონ ქვემოთ მოცემული დავალებები.

სავარჯიშო 1 — ელემენტების მოძიება ID-ით

შექმენით HTML გვერდი, რომელშიც იქნება:

- ერთი `h1` ელემენტი, რომელსაც ექნება `id="title"`.
- ერთი `h2` ელემენტი, რომელსაც ექნება `id="subtitle"`.
- ერთი `p` ელემენტი, რომელსაც ექნება `id="description"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ სათაური მისი `id`-ის მიხედვით.
2. მოძებნეთ ქვესათაური მისი `id`-ის მიხედვით.

3. მოძებნეთ პარაგრაფი მისი `id`-ის მიხედვით.
4. სამივე მოძიებული ელემენტი გამოიტანეთ კონსოლში.
5. შეამოწმეთ, რა მოხდება, თუ მოძებნით არარსებული `id`-ის მქონე ელემენტს.

სავარჯიშო 2 — მოსწავლეების სია

შექმენით HTML გვერდი, რომელზეც იქნება 5 პარაგრაფი.

ყველა პარაგრაფში ჩაწერეთ სხვადასხვა მოსწავლის სახელი და ყველას მიანიჭეთ `class="student"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ ყველა ელემენტი, რომელსაც აქვს `student` კლასი.
2. გამოიტანეთ კონსოლში ყველა მოძიებული ელემენტი.
3. გამოიტანეთ მოსწავლეების რაოდენობა.
4. ცალკე გამოიტანეთ პირველი და ბოლო მოსწავლის ელემენტები.
5. `for` ციკლის გამოყენებით სათითაოდ გამოიტანეთ ხუთივე ელემენტი.

სავარჯიშო 3 — საყვარელი ფილმები

შექმენით HTML გვერდი, რომელზეც იქნება სათაური და დაულაგებელი სია (`ul`).

სიაში დაამატეთ 6 საყვარელი ფილმი `li` ელემენტების სახით.

JavaScript-ის გამოყენებით:

1. `getElementsByTagName()` მეთოდით მოძებნეთ ყველა `li` ელემენტი.
2. გამოიტანეთ სიის ელემენტების რაოდენობა.
3. კონსოლში გამოიტანეთ მეორე და მეხუთე ელემენტები.
4. `for` ციკლით გამოიტანეთ ყველა `li` ელემენტი.
5. მოიძიეთ ყველა `ul` ელემენტი და გამოიტანეთ მათი რაოდენობა.

სავარჯიშო 4 — სკოლის ვებგვერდი

შექმენით სკოლის საინფორმაციო HTML გვერდი.

გვერდზე უნდა იყოს:

- სკოლის სახელი `h1` ელემენტში, რომელსაც ექნება `id="school-name"`.
- სამი `h2` სათაური.
- ოთხი პარაგრაფი, რომელთაც ექნებათ `class="info"`.
- მოსწავლეების სია, რომელშიც იქნება მინიმუმ 5 `li` ელემენტი.

- სამი `div` ელემენტი, რომელთაც ექნებათ `class="card"`.

JavaScript-ის გამოყენებით:

1. მოძებნეთ სკოლის სახელი `id`-ის დახმარებით.
2. მოძებნეთ ყველა საინფორმაციო პარაგრაფი კლასის მიხედვით.
3. მოძებნეთ ყველა `h2` ელემენტი ტეგის მიხედვით.
4. დაითვალეთ რამდენი `li` ელემენტია გვერდზე.
5. მოძებნეთ ყველა `card` კლასის მქონე ელემენტი.
6. კონსოლში გამოიტანეთ ყველა კატეგორიის ელემენტების რაოდენობა.
7. `for` ციკლის გამოყენებით ცალ-ცალკე გამოიტანეთ ყველა პარაგრაფი და ყველა ბარათი.